

```

# 8 Puzzle using Iterative Deepening Search

def dls(state, goal, path, visited, depth):

    # Goal test
    if state == goal:
        return path

    # Depth limit reached
    if depth == 0:
        return None

    # Position of blank
    blank = state.index(0)

    # Possible moves
    moves = {
        0: [1, 3],
        1: [0, 2, 4],
        2: [1, 5],
        3: [0, 4, 6],
        4: [1, 3, 5, 7],
        5: [2, 4, 8],
        6: [3, 7],
        7: [4, 6, 8],
        8: [5, 7]
    }

    for new_position in moves[blank]:

        new_state = list(state)

        # Swap blank
        new_state[blank], new_state[new_position] = \
            new_state[new_position], new_state[blank]

        new_state = tuple(new_state)

        # Avoid states already in current path
        if new_state not in visited:

            visited.add(new_state)

            result = dls(
                new_state,
                goal,
                path + [new_state],
                visited,
                depth - 1
            )

            if result is not None:
                return result

            visited.remove(new_state)

    return None

# Input
print("Enter initial state:")
initial = tuple(map(int, input().split()))

print("Enter goal state:")
goal = tuple(map(int, input().split()))

# Iterative Deepening
for depth in range(31):

    visited = {initial}

    path = dls(
        initial,
        goal,

```

```

        [initial],
        visited,
        depth
    )

    if path is not None:

        print("\nSolution found!")
        print("Depth:", depth)
        print("Path Cost:", len(path) - 1)

        print("\nPath:")

        for state in path:
            print(state)

        break

    else:
        print("No solution found.")

```

Enter initial state:

1 2 3 0 4 6 7 5 8

Enter goal state:

1 2 3 4 5 6 7 8 0

Solution found!

Depth: 3

Path Cost: 3

Path:

(1, 2, 3, 0, 4, 6, 7, 5, 8)

(1, 2, 3, 4, 0, 6, 7, 5, 8)

(1, 2, 3, 4, 5, 6, 7, 0, 8)

(1, 2, 3, 4, 5, 6, 7, 8, 0)

```

import heapq

class PuzzleNode:
    def __init__(self, state, g_cost, h_cost, parent=None):
        self.state = state          # 2D tuple representing the 3x3 grid
        self.g_cost = g_cost        # Distance from start node (moves made)
        self.h_cost = h_cost        # Heuristic distance to goal
        self.f_cost = g_cost + h_cost # Total cost f(n) = g(n) + h(n)
        self.parent = parent        # Reference to parent node to reconstruct path

    # Define comparison operator for the priority queue (min-heap)
    def __lt__(self, other):
        return self.f_cost < other.f_cost

def get_manhattan_distance(state, goal_state):
    """Calculates the sum of absolute distances of tiles from their goal positions."""
    distance = 0
    # Create a mapping of value -> (row, col) for the goal state
    goal_positions = {}
    for r in range(3):
        for c in range(3):
            goal_positions[goal_state[r][c]] = (r, c)

    for r in range(3):
        for c in range(3):
            val = state[r][c]
            if val != 0: # Ignore the blank tile (0)
                target_r, target_c = goal_positions[val]
                distance += abs(r - target_r) + abs(c - target_c)
    return distance

def get_neighbors(state):
    """Finds all valid adjacent states by moving the blank tile (0)."""
    neighbors = []
    # Find the position of the blank tile (0)
    blank_r, blank_c = 0, 0
    for r in range(3):
        for c in range(3):
            if state[r][c] == 0:
                blank_r, blank_c = r, c
                break

```

```

# Possible movements: Up, Down, Left, Right
moves = [(-1, 0), (1, 0), (0, -1), (0, 1)]

for dr, dc in moves:
    new_r, new_c = blank_r + dr, blank_c + dc
    if 0 <= new_r < 3 and 0 <= new_c < 3:
        # Convert tuple of tuples to mutable list of lists to perform swap
        new_state = [list(row) for row in state]
        # Swap blank space with adjacent tile
        new_state[blank_r][blank_c], new_state[new_r][new_c] = new_state[new_r][new_c], new_state[blank_r][blank_c]
        # Convert back to immutable tuple of tuples for hashing/set storage
        neighbors.append(tuple(tuple(row) for row in new_state))

return neighbors

def solve_8_puzzle(initial_state, goal_state):
    """Solves the 8-puzzle problem using A* algorithm and prints metrics for each state."""
    # Priority Queue for tracking open nodes
    open_list = []
    # Set for tracking visited states
    closed_set = set()

    # Initialize start node
    start_h = get_manhattan_distance(initial_state, goal_state)
    start_node = PuzzleNode(initial_state, 0, start_h)

    heapq.heappush(open_list, start_node)

    while open_list:
        current_node = heapq.heappop(open_list)

        # If goal is reached, reconstruct the path
        if current_node.state == goal_state:
            path = []
            while current_node:
                path.append(current_node)
                current_node = current_node.parent
            return path[::-1] # Return path from start to goal

        closed_set.add(current_node.state)

        # Generate and evaluate child states
        for neighbor_state in get_neighbors(current_node.state):
            if neighbor_state in closed_set:
                continue

            g_cost = current_node.g_cost + 1
            h_cost = get_manhattan_distance(neighbor_state, goal_state)
            neighbor_node = PuzzleNode(neighbor_state, g_cost, h_cost, current_node)

            # Check if neighbor is already in open list with a lower cost
            # (Simplification for standard min-heap pattern in 8-puzzle)
            heapq.heappush(open_list, neighbor_node)

    return None

def print_solution(path):
    """Formats and prints the puzzle path along with g(n), h(n), and f(n)."""
    if not path:
        print("No solution found.")
        return

    print(f"Optimal solution found in {len(path) - 1} moves!\n")
    for index, node in enumerate(path):
        print(f"--- Step {index} ---")
        for row in node.state:
            # Format row visualization replacing '0' with a blank space indicator '_'
            print(" ".join(str(tile) if tile != 0 else "_" for tile in row))
        print(f"g(n) = {node.g_cost} (moves from start)")
        print(f"h(n) = {node.h_cost} (Manhattan distance to goal)")
        print(f"f(n) = {node.f_cost} (total cost evaluation)")
        print("-" * 20 + "\n")

# --- CONFIGURATION & EXECUTION ---
if __name__ == "__main__":
    # 0 represents the empty grid block. Custom grids can be supplied below.

```

```
# Note: Use tuple structure to ensure immutability inside code sets.
```

```
# A standard configuration guaranteed to be solvable
```

```
initial = (
    (1, 2, 3),
    (0, 4, 6),
    (7, 5, 8)
)
```

```
goal = (
    (1, 2, 3),
    (4, 5, 6),
    (7, 8, 0)
)
```

```
solution_path = solve_8_puzzle(initial, goal)
print_solution(solution_path)
```

Optimal solution found in 3 moves!

--- Step 0 ---

```
1 2 3
_ 4 6
7 5 8
g(n) = 0 (moves from start)
h(n) = 3 (Manhattan distance to goal)
f(n) = 3 (total cost evaluation)
-----
```

--- Step 1 ---

```
1 2 3
4 _ 6
7 5 8
g(n) = 1 (moves from start)
h(n) = 2 (Manhattan distance to goal)
f(n) = 3 (total cost evaluation)
-----
```

--- Step 2 ---

```
1 2 3
4 5 6
7 _ 8
g(n) = 2 (moves from start)
h(n) = 1 (Manhattan distance to goal)
f(n) = 3 (total cost evaluation)
-----
```

--- Step 3 ---

```
1 2 3
4 5 6
7 8 _
g(n) = 3 (moves from start)
h(n) = 0 (Manhattan distance to goal)
f(n) = 3 (total cost evaluation)
-----
```

```
import heapq

class PuzzleNode:
    def __init__(self, state, g_cost, h_cost, parent=None):
        self.state = state          # 2D tuple representing the 3x3 grid
        self.g_cost = g_cost        # Distance from start node (moves made)
        self.h_cost = h_cost        # Heuristic distance to goal
        self.f_cost = g_cost + h_cost # Total cost f(n) = g(n) + h(n)
        self.parent = parent        # Reference to parent node to reconstruct path

    def __lt__(self, other):
        return self.f_cost < other.f_cost

def get_manhattan_distance(state, goal_state):
    """Calculates the sum of absolute distances of tiles from their goal positions."""
    distance = 0
    goal_positions = {}
    for r in range(3):
        for c in range(3):
            goal_positions[goal_state[r][c]] = (r, c)

    for r in range(3):
```

```

        for c in range(3):
            val = state[r][c]
            if val != 0: # Ignore the blank tile (0)
                target_r, target_c = goal_positions[val]
                distance += abs(r - target_r) + abs(c - target_c)
        return distance

def get_neighbors(state):
    """Finds all valid adjacent states by moving the blank tile (0)."""
    neighbors = []
    blank_r, blank_c = 0, 0
    for r in range(3):
        for c in range(3):
            if state[r][c] == 0:
                blank_r, blank_c = r, c
                break

    moves = [(-1, 0), (1, 0), (0, -1), (0, 1)] # Up, Down, Left, Right

    for dr, dc in moves:
        new_r, new_c = blank_r + dr, blank_c + dc
        if 0 <= new_r < 3 and 0 <= new_c < 3:
            new_state = [list(row) for row in state]
            new_state[blank_r][blank_c], new_state[new_r][new_c] = new_state[new_r][new_c], new_state[blank_r][blank_c]
            neighbors.append(tuple(tuple(row) for row in new_state))

    return neighbors

def solve_8_puzzle(initial_state, goal_state):
    """Solves the 8-puzzle problem using A* algorithm."""
    open_list = []
    closed_set = set()

    start_h = get_manhattan_distance(initial_state, goal_state)
    start_node = PuzzleNode(initial_state, 0, start_h)
    heapq.heappush(open_list, start_node)

    while open_list:
        current_node = heapq.heappop(open_list)

        if current_node.state == goal_state:
            path = []
            while current_node:
                path.append(current_node)
                current_node = current_node.parent
            return path[::-1]

        closed_set.add(current_node.state)

        for neighbor_state in get_neighbors(current_node.state):
            if neighbor_state in closed_set:
                continue

            g_cost = current_node.g_cost + 1
            h_cost = get_manhattan_distance(neighbor_state, goal_state)
            neighbor_node = PuzzleNode(neighbor_state, g_cost, h_cost, current_node)
            heapq.heappush(open_list, neighbor_node)

    return None

def print_solution(path):
    """Formats and prints the puzzle path along with g(n), h(n), and f(n)."""
    if not path:
        print("\n[!] No solution found. The puzzle may be unsolvable from this configuration.")
        return

    print(f"\n=====")
    print(f" Optimal Solution Found in {len(path) - 1} Moves!")
    print(f"=====\\n")
    for index, node in enumerate(path):
        print(f"--- Step {index} ---")
        for row in node.state:
            print(" ".join(str(tile) if tile != 0 else "_" for tile in row))
        print(f"g(n) = {node.g_cost} (moves from start)")
        print(f"h(n) = {node.h_cost} (Manhattan distance to goal)")
        print(f"f(n) = {node.f_cost} (total state cost)")
        print("-" * 25 + "\\n")

```

```

def get_user_matrix(matrix_name):
    """Prompts the user to input a valid 3x3 grid matrix configuration."""
    print(f"\n--- Enter the configuration for the {matrix_name} ---")
    print("Use numbers 1-8 for tiles, and 0 or '_' for the blank space.")
    print("Separate numbers in a row by spaces (e.g., '1 2 3').")

    while True:
        try:
            grid = []
            for i in range(3):
                row_input = input(f"Enter row {i+1}: ").strip().replace('_', '0')
                row_parsed = [int(x) for x in row_input.split()]
                if len(row_parsed) != 3:
                    raise ValueError("Each row must contain exactly 3 numbers.")
                grid.append(tuple(row_parsed))

            full_matrix = tuple(grid)

            # Validate if the matrix contains exactly numbers from 0 to 8
            flattened = [item for row in full_matrix for item in row]
            if sorted(flattened) != list(range(9)):
                print("[Error] Invalid configuration! The board must contain numbers from 0 to 8 with no duplicates.")
                continue

            return full_matrix
        except ValueError as e:
            print(f"[Error] Invalid input: {e}. Please re-enter the entire grid matrix.")

# --- MAIN EXECUTION ---
if __name__ == "__main__":
    print("=====")
    print("8-Puzzle Solver using A* Search Algorithm")
    print("=====")

    # Take runtime input from the user
    initial_state = get_user_matrix("INITIAL STATE")
    goal_state = get_user_matrix("GOAL STATE")

    print("\n[Thinking] Processing search graph path...")
    solution_path = solve_8_puzzle(initial_state, goal_state)
    print_solution(solution_path)

```

```

=====
8-Puzzle Solver using A* Search Algorithm
=====

--- Enter the configuration for the INITIAL STATE ---
Use numbers 1-8 for tiles, and 0 or '_' for the blank space.
Separate numbers in a row by spaces (e.g., '1 2 3').
Enter row 1: 2 8 3 1 6 4 0 7 5
[Error] Invalid input: Each row must contain exactly 3 numbers.. Please re-enter the entire grid matrix.
Enter row 1: 2 8 3
Enter row 2: 1 6 4
Enter row 3: 0 7 5

--- Enter the configuration for the GOAL STATE ---
Use numbers 1-8 for tiles, and 0 or '_' for the blank space.
Separate numbers in a row by spaces (e.g., '1 2 3').
Enter row 1: 1 2 3
Enter row 2: 8 0 4
Enter row 3: 7 6 5

[Thinking] Processing search graph path...

=====
Optimal Solution Found in 6 Moves!
=====

--- Step 0 ---
2 8 3
1 6 4
_ 7 5
g(n) = 0 (moves from start)
h(n) = 6 (Manhattan distance to goal)
f(n) = 6 (total state cost)
-----

--- Step 1 ---
2 8 3

```

```
1 6 4
7 _ 5
g(n) = 1 (moves from start)
h(n) = 5 (Manhattan distance to goal)
f(n) = 6 (total state cost)
-----

--- Step 2 ---
2 8 3
1 _ 4
7 6 5
g(n) = 2 (moves from start)
h(n) = 4 (Manhattan distance to goal)
f(n) = 6 (total state cost)
-----

--- Step 3 ---
2 _ 3
1 8 4
7 6 5
g(n) = 3 (moves from start)
```